Algorithm Engineering
Woche 2, Aufgabe 2

Jan Vogt

Die Abbildung „Performance gains after Moore's law ends“

Die Abbildung auf der ersten Seite des Papers „There's plenty of room at the Top“ von Leiserson et al. (Science, 2020) fasst die Aussage des ganzen Papers zusammen. Sie besteht aus zwei Bereichen. Oben steht ein Kasten „The Top“ mit den drei Spalten Software, Algorithms und Hardware architecture. Für jede Spalte gibt es ein Bild (Technology), eine Möglichkeit, wie man dort noch Leistung herausholen kann (Opportunity), und zwei Beispiele (Examples). Unten steht „The Bottom“, gezeichnet als Trichter aus Streifen, die nach unten immer schmaler werden. So wie ich das verstehe, soll das zeigen, dass dort immer weniger Platz für Verbesserungen übrig ist.

The Bottom

Mit Bottom ist die Halbleitertechnik gemeint, also das Verkleinern der Transistoren nach Moore's Law. Über 50 Jahre kam fast die ganze Leistungssteigerung von hier, und zwar automatisch: Ein Programm wurde auf einem neuen Prozessor schneller, ohne dass man etwas daran ändern musste. Laut Paper ist dieser Platz aber fast aufgebraucht. Intel hat zum Beispiel die 10-nm-Technik erst 2019 statt 2016 geschafft, und bei ungefähr 5 nm wird die Verkleinerung wohl aufhören, weil sie danach kaum noch etwas bringt und immer teurer wird. Deshalb sagt die Bildunterschrift, dass sich der bisherige Trend umdreht und die Gewinne in Zukunft vor allem aus dem Top kommen.

Die drei Spalten im Top

Software. Die Möglichkeit ist Software performance engineering, also Programme so umzubauen, dass sie schneller laufen. Das heißt einerseits Software bloat entfernen, weil Software bisher eher schnell fertig als schnell sein sollte, und andererseits die Software an die Hardware anpassen, zum Beispiel an mehrere Kerne, Vektoreinheiten und Caches. Im Paper wird eine 4096×4096 Matrixmultiplikation in Python, die etwa 7 Stunden braucht, Schritt für Schritt bis auf 0,41 Sekunden beschleunigt. Das ist mehr als 60.000 mal schneller.

Algorithms. Bessere Algorithmen lösen ein Problem grundsätzlich schneller. Beim Maximum-Flow-Problem haben neue Algorithmen über 38 Jahre fast so viel gebracht wie Moore's Law, seit 20 Jahren aber nichts mehr. Fortschritt kommt laut Paper vor allem durch neue Problembereiche wie Machine Learning (New problem domains) und durch neue Maschinenmodelle, weil die meisten Algorithmen noch für einen Prozessor entworfen wurden, der nur eine Operation gleichzeitig macht und bei dem jeder Speicherzugriff gleich lange dauert (New machine models).

Hardware architecture. Wenn es nicht mehr Transistoren gibt, muss man die vorhandenen besser nutzen (Hardware streamlining). Bei Processor simplification werden aufwendige Kerne durch einfachere ersetzt, damit mehr Kerne auf den Chip passen. Das lohnt sich aber nur, wenn das Programm genug Parallelität hat. Bei Domain specialization wird Hardware für einen bestimmten Bereich gebaut, zum Beispiel GPUs oder die TPU von Google für Deep Learning.

Gesamtaussage

Auch nach dem Ende von Moore's Law kann man Computer also noch schneller machen, nur nicht mehr automatisch. Beim Top muss man für jedes Problem gezielt etwas ändern, und die Gewinne kommen laut Paper ungleichmäßig und werden kleiner, je besser ein Problem schon erforscht ist.

Das ist mir auch im Zusammenhang mit der Vorlesung aufgefallen: Die Spalte Software ist im Grunde genau das, was wir in Algorithm Engineering machen. Bei der Matrixmultiplikation im Paper kommen die großen Sprünge durch Parallelisierung, Cache-Ausnutzung und Vektorisierung, also genau die Themen aus der Kursübersicht. Außerdem steht im Bild für Software die Folge `01010011 01100011 01101001 01100101 01101110 01100011 01100101 00000000`. Liest man die Bytes als ASCII wie in Kapitel 1 von CS:APP, ergibt das das Wort „Science“ mit einem Null-Byte am Ende, also genau wie ein String in C.